CSC322 – OPERATING SYSTEMS

LAB NO. 05 – Student Practical Record

Managing Processes, and Writing, Compiling & Executing C++ on Linux

Every command from the lab manual, run on a real Linux system, with its actual output

How to Use This Document

Each step below shows the command that was typed, followed by the real output the system produced. Use it to check your own results while doing the lab.

```
$ command typed at the terminal
output printed by the system
C++ source code you save in a .cpp file
```

Operating system	Ubuntu 24.04 (Linux kernel 6.18)
Compiler	g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
ps version	procps-ng 4.0.4
User account used	root (the lab text uses the user name ubuntu)

Important notes before you compare your output

- **Your numbers will differ.** PIDs, times, memory sizes and the random prime count will be different on your computer. The format and the meaning of the columns will be the same.
- **Server vs desktop.** This output comes from a minimal Linux server with no desktop, so ps shows many kernel threads. On your Ubuntu desktop you will also see bash, gnome and other programs.
- **top.** top is interactive and cannot be pasted as a live screen, so it was run in batch mode (top -b -n 1), which prints one refresh to the screen. On your computer, run plain top -d 0.5 and press q to quit.
- **Substitutions.** gedit, pstree and xload are not installed on the system used to record these results, and there is no graphical screen. A background sleep 300 command was used in place of gedit, and ps -ejH (an indented process tree) was shown in place of pstree. The job-control commands behave identically for gedit. Ctrl+Z was simulated with kill -TSTP, which is the same signal that Ctrl+Z sends.
- **ps -x -F.** The manual's command ps -x -F gives an error on newer ps versions. The working form is ps x -F (shown in Step 3).
- **Trimmed lists.** Long listings are cut with | head -N so they fit on the page. The part of the command after | head only limits how many lines are shown.

PART A – Process Management

Step 1 – ps: processes in the current shell

ps gives a snapshot of processes. PID is the unique Process ID, TTY the terminal, TIME the CPU time used and CMD the program name.

```
$ ps
  PID TTY          TIME CMD
    1 ?        00:00:00 process_api
    2 ?        00:00:00 kthreadd
    3 ?        00:00:00 pool_workqueue_release
    4 ?        00:00:00 kworker/R-rcu_gp
    5 ?        00:00:00 kworker/R-sync_wq
    6 ?        00:00:00 kworker/R-kvfree_rcu_reclaim
    7 ?        00:00:00 kworker/R-slub_flushwq
    8 ?        00:00:00 kworker/R-netns
    9 ?        00:00:00 kworker/0:0-events
[... 36 more lines omitted ...]
   55 ?        00:00:00 rclone-filestor
   90 ?        00:00:00 sh
   92 ?        00:00:00 ps
```

Observation: The first column is the PID. PID 1 is the very first process started by the system (the init process). Here it appears as process_api; on a normal Ubuntu machine it is systemd. ps itself also appears in the list, with its own PID, because it was running when the snapshot was taken. Note: this system has no terminal attached, so ps listed every process. In a normal terminal window plain ps lists only your shell (bash) and ps itself.

Step 2 – ps -A and ps -e: every active process

```
$ ps -A | head -15
  PID TTY          TIME CMD
    1 ?        00:00:00 process_api
    2 ?        00:00:00 kthreadd
    3 ?        00:00:00 pool_workqueue_release
    4 ?        00:00:00 kworker/R-rcu_gp
    5 ?        00:00:00 kworker/R-sync_wq
    6 ?        00:00:00 kworker/R-kvfree_rcu_reclaim
    7 ?        00:00:00 kworker/R-slub_flushwq
    8 ?        00:00:00 kworker/R-netns
    9 ?        00:00:00 kworker/0:0-events
   10 ?        00:00:00 kworker/0:0H-events_highpri
   11 ?        00:00:00 kworker/0:1-virtio_vsock
   12 ?        00:00:00 kworker/u4:0-events_unbound
   13 ?        00:00:00 kworker/R-mm_percpu_wq
   14 ?        00:00:00 ksoftirqd/0
```

Observation: Both -A and -e list all processes on the system. Only the first 15 lines are shown here.

```
$ ps -e | wc -l
50
```

Observation: Number of lines = number of processes + 1 header line, so 49 processes were running at that moment.

Step 3 – ps -x and ps -x -F: your own processes, full format

```
$ ps -x | head -12
  PID TTY      STAT   TIME COMMAND
    1 ?        SLl    0:00 /process_api --firecracker-init --addr 0.0.0.0:2024 --max-
ws-buffer-size 32768 --block-local-connections --listen-vsock-port 2024 --log-vsock-
port 5002
    2 ?        S      0:00 [kthreadd]
    3 ?        S      0:00 [pool_workqueue_release]
    4 ?        I<     0:00 [kworker/R-rcu_gp]
    5 ?        I<     0:00 [kworker/R-sync_wq]
    6 ?        I<     0:00 [kworker/R-kvfree_rcu_reclaim]
    7 ?        I<     0:00 [kworker/R-slub_flushwq]
    8 ?        I<     0:00 [kworker/R-netns]
    9 ?        I      0:00 [kworker/0:0-events]
   10 ?        I<     0:00 [kworker/0:0H-events_highpri]
   11 ?        I      0:00 [kworker/0:1-virtio_vsock]
```

Observation: The STAT column shows the process state (S = sleeping, I = idle kernel thread, < = high priority, l = multithreaded).

```
$ ps -x -F | head -8
error: must set personality to get -x option

Usage:
 ps [options]

 Try 'ps --help <simple|list|output|threads|misc|all>'
  or 'ps --help <s|l|o|t|m|a>'
 for additional help text.

For more details see ps(1).
```

Observation: On this version of ps, the dash form -x -F is rejected.

```
$ ps x -F | head -4
UID        PID  PPID  C    SZ   RSS PSR STIME TTY      STAT   TIME CMD
root         1     0  2  4631  5596   0 04:52 ?        SLl    0:00 /process_api --
firecracker-init --addr 0.0.0.0:2024 --max-ws-buffer-size 32768 --block-local-
connections --listen-vsock-port 2024 --log-vsock-port 5002
root         2     0  0     0     0   0 04:52 ?        S      0:00 [kthreadd]
root         3     2  0     0     0   0 04:52 ?        S      0:00
[pool_workqueue_release]
```

Observation: Without the dash, x works together with -F. The extra columns are UID, PPID (parent PID), C (CPU use), SZ and RSS (memory), PSR (CPU number) and STIME (start time).

Step 4 – ps -u USER -F: processes of a given user

The manual uses the user name ubuntu. Replace it with the user you are logged in as (the output here uses root).

```
$ ps -u root -F | head -8
UID        PID  PPID  C    SZ   RSS PSR STIME TTY          TIME CMD
root         1     0  5  4631  5596   0 04:52 ?        00:00:00 /process_api --
firecracker-init --addr 0.0.0.0:2024 --max-ws-buffer-size 32768 --block-local-
connections --listen-vsock-port 2024 --log-vsock-port 5002
root         2     0  0     0     0   0 04:52 ?        00:00:00 [kthreadd]
root         3     2  0     0     0   0 04:52 ?        00:00:00
[pool_workqueue_release]
root         4     2  0     0     0   0 04:52 ?        00:00:00 [kworker/R-rcu_gp]
root         5     2  0     0     0   0 04:52 ?        00:00:00 [kworker/R-sync_wq]
root         6     2  0     0     0   0 04:52 ?        00:00:00 [kworker/R-
kvfree_rcu_reclaim]
root         7     2  0     0     0   0 04:52 ?        00:00:00 [kworker/R-
slub_flushwq]
```

Observation: PPID shows the parent. kthreadd (PID 2) is the parent of most kernel threads, so their PPID is 2.

Step 5 – ps -p PID: one specific process

```
$ ps -p 1
  PID TTY          TIME CMD
    1 ?        00:00:00 process_api
```

```
$ ps -p 1 -F
UID        PID  PPID  C    SZ   RSS PSR STIME TTY          TIME CMD
root         1     0  5  4631  5596   0 04:52 ?        00:00:00 /process_api --
firecracker-init --addr 0.0.0.0:2024 --max-ws-buffer-size 32768 --block-local-
connections --listen-vsock-port 2024 --log-vsock-port 5002
```

Observation: PID 1 has PPID 0, which means it has no parent. It is the ancestor of every other process.

Step 6 – ps -r, ps -L and ps -G

```
$ ps -r
  PID TTY      STAT   TIME COMMAND
```

Observation: ps -r shows only processes in state R (running). The system was idle when the snapshot was taken, so no process matched and only the header line was printed. On a busy system you will see some lines.

```
$ ps -L | head -8
  PID   LWP TTY          TIME CMD
    1     1 ?        00:00:00 process_api
    1    47 ?        00:00:00 vsock-console
    1    48 ?        00:00:00 tokio-rt-worker
    1    52 ?        00:00:00 idle-reclaim
    1    53 ?        00:00:00 sbx-telemetry-s
    2     2 ?        00:00:00 kthreadd
    3     3 ?        00:00:00 pool_workqueue_release
```

Observation: LWP is the thread ID. PID 1 has several threads (LWP 1, 47, 48, 52, 53), each shown on its own line.

```
$ ps -G root | head -8
  PID TTY          TIME CMD
    1 ?        00:00:00 process_api
    2 ?        00:00:00 kthreadd
    3 ?        00:00:00 pool_workqueue_release
    4 ?        00:00:00 kworker/R-rcu_gp
    5 ?        00:00:00 kworker/R-sync_wq
    6 ?        00:00:00 kworker/R-kvfree_rcu_reclaim
    7 ?        00:00:00 kworker/R-slub_flushwq
```

Observation: Lists processes whose group is root.

Step 7 – top: live view of processes

Plain top keeps refreshing the screen until you press q. -d 0.5 sets the refresh delay to half a second and -u limits the view to one user. The commands below use -b -n 1 (batch mode, one refresh) so the output can be shown here.

```
$ top -b -n 1 -d 0.5 | head -15
top - 04:52:22 up 0 min,  0 user,  load average: 0.00, 0.00, 0.00
Tasks:  50 total,   1 running,  49 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  9.1 sy,  0.0 ni, 90.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3999.3 total,   3900.0 free,    189.2 used,     57.9 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   3810.1 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root      20   0   18524   5596   5212 S   0.0   0.1   0:00.79 process_a+
    2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
    3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_work+
    4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R+
    5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R+
    6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R+
    7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R+
    8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R+
```

Observation: The top part is a summary: uptime, load average, number of tasks, CPU and memory use. Below it is the process table with PR (priority) and NI (nice value). Normal processes have PR 20 and NI 0. Kernel threads shown as kworker/R+ have PR 0 and NI -20, which is the highest priority.

```
$ top -b -n 1 -u root | head -12
top - 04:52:22 up 0 min,  0 user,  load average: 0.00, 0.00, 0.00
Tasks:  50 total,   1 running,  49 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3999.3 total,   3900.0 free,    189.0 used,     58.2 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   3810.3 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root      20   0   18524   5596   5212 S   0.0   0.1   0:00.79 process_a+
    2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
    3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_work+
    4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R+
    5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R+
```

Observation: The same display, filtered to the user given after -u.

Step 8 – process tree (pstree)

pstree prints the process hierarchy as a tree. It is not installed on this system, so ps -ejH is used, which shows the same parent–child structure using indentation. On your Ubuntu machine simply run pstree.

```
$ which pstree gedit xload
(no output – these programs are not installed on this system)
```

```
$ ps -ejH | head -12
  PID  PGID   SID TTY          TIME CMD
    2     0     0 ?        00:00:00 kthreadd
    3     0     0 ?        00:00:00   pool_workqueue_release
    4     0     0 ?        00:00:00   kworker/R-rcu_gp
    5     0     0 ?        00:00:00   kworker/R-sync_wq
    6     0     0 ?        00:00:00   kworker/R-kvfree_rcu_reclaim
    7     0     0 ?        00:00:00   kworker/R-slub_flushwq
    8     0     0 ?        00:00:00   kworker/R-netns
    9     0     0 ?        00:00:00   kworker/0:0-events
   10     0     0 ?        00:00:00   kworker/0:0H-events_highpri
   11     0     0 ?        00:00:00   kworker/0:1-events_power_efficient
   12     0     0 ?        00:00:00   kworker/u4:0-events_unbound
```

Observation: The indented lines are child processes of the line above (for example the pool_workqueue_release and kworker lines are children of kthreadd).

Step 9 – Foreground, background and job control

gedit & starts a program in the background so the prompt comes back immediately. On the machine used for this record a background sleep 300 stands in for gedit.

```
$ sleep 300 &   (stand-in for gedit &)
(the shell returns to the prompt immediately; the job runs in the background)
```

```
$ jobs
[1]+  Running                 sleep 300 &
```

Observation: Job number [1] is Running in the background (+ marks the current job).

```
$ Ctrl+Z  (simulated: kill -TSTP %1)
(Ctrl+Z suspends the foreground job)
```

```
$ jobs
[1]+  Stopped                 sleep 300
```

Observation: The job is now Stopped (suspended). It is not using the CPU.

```
$ bg
[1]+ sleep 300 &
```

Observation: bg resumes the stopped job in the background.

```
$ jobs
[1]+  Running                 sleep 300 &
```

Observation: The job is Running again.

To bring the job back to the foreground type fg. The terminal is then occupied by the job until it finishes, or until you press Ctrl+Z (suspend) or Ctrl+C (interrupt). Because fg takes over the screen, it is not shown as a recorded output here.

```
$ kill %1
[1]+  Terminated              sleep 300
```

Observation: Ends the demonstration job.

```
$ jobs
(no output – there are no jobs left)
```

Step 10 – nice and renice (process priority)

The nice value (NI) ranges from -20 (highest priority) to +19 (lowest). The default is 0. On this system ps -l shows PRI = 80 + NI, so nice 0 gives PRI 80 and nice 10 gives PRI 90.

```
$ nice
0
```

Observation: With no arguments, nice prints the current niceness, which is 0 by default.

```
$ nice -n 10 sleep 300 &
(the program starts in the background with a nice value of 10)
```

```
$ ps -l | grep -E 'PID|sleep'
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S     0   136   123  0  90  10 -   676 hrtime ?        00:00:00 sleep
```

Observation: NI = 10 and PRI = 90.

```
$ renice -n 5 -p 136
136 (process ID) old priority 10, new priority 5
```

```
$ ps -l | grep -E 'PID|sleep'
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S     0   136   123  0  85   5 -   676 hrtime ?        00:00:00 sleep
```

Observation: After renice the nice value is 5 and PRI = 85.

```
$ ps -o pid,pri,ni,cmd -p 136
  PID PRI  NI CMD
  136  14   5 sleep 300
```

Observation: Careful: ps -o pri uses a different scale and shows 14 for the same process. Always use ps -l when you want to compare PRI with the manual.

Activity 2 – Start a program with priority 90, then reset it to 65

Using ps -l, PRI = 80 + NI. So priority 90 means NI = 10, and priority 65 means NI = -15.

```
$ nice -n 10 sleep 300 &
(started in the background with nice value 10)
```

```
$ ps -l -p 147
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S     0   147   146  0  90  10 -   676 hrtime ?        00:00:00 sleep
```

Observation: PRI = 90 (NI = 10).

```
$ renice -n -15 -p 147
147 (process ID) old priority 10, new priority -15
```

Observation: Lowering the number raises the priority. Only the superuser (root) may set a negative nice value. A normal user must use sudo renice -n -15 -p PID.

```
$ ps -l -p 147
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S     0   147   146  0  65 -15 -   676 hrtime ?        00:00:00 sleep
```

Observation: PRI = 65 (NI = -15). The priority was changed from 90 to 65.

Step 11 – vmstat

vmstat 5 would print a new line every 5 seconds until you press Ctrl+C. Here it is run with a 1-second interval and 3 samples.

```
$ vmstat 1 3
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 0  0      0 3993448   3816  55776    0    0  1083     1  130    2  1  8 90  1  0  0
 0  0      0 3993436   3816  55844    0    0     0     0   27   39  0  0 100  0  0  0
 0  0      0 3993436   3816  55844    0    0     0     0   44   69  0  0 100  0  0  0
```

Observation: Columns r and b are runnable and blocked processes; free is free memory (KB); si/so show swapping; us, sy and id show CPU time in user mode, kernel mode and idle. The first line is an average since boot and the next lines show current activity. tload and xload (load-average displays) need a live terminal or screen, so they are not recorded here.

PART B – C++ on Linux

Step 12 – Check the compiler

```
$ g++ --version
g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
```

Observation: g++ is installed. If you get "command not found", install it with sudo apt-get update followed by sudo apt install g++.

Step 13 – Compile and run hello.cpp

hello.cpp

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World!";
    return 0;
}
```

```
$ g++ hello.cpp
(no output means the compilation succeeded)
```

```
$ ls
a.out
hello.cpp
```

Observation: Without the -o option the executable is named a.out.

```
$ ./a.out
Hello World!
```

Observation: The program does not print a new line, so the next prompt would appear directly after the text.

```
$ g++ hello.cpp -o hello
(no output means the compilation succeeded)
```

```
$ ./hello
Hello World!
```

```
$ hello   (forgot ./)
bash: line 1: hello: command not found
```

Observation: Common mistake: without ./ Linux looks for hello in the system folders, not the current folder.

Step 14 – Command-line arguments (avg.cpp)

argc is the number of arguments including the program name. argv[0] is the program name, argv[1] is the first value, and so on. For ./avg 10 20 30, argc = 4.

avg.cpp

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char *argv[])
{
    double sum = 0;

    for (int i = 1; i < argc; i++)
```

```
        sum += atof(argv[i]);

    if (argc > 1)
        cout << "Average = " << sum / (argc - 1) << endl;

    return 0;
}
```

```
$ g++ avg.cpp -o avg
(no output means the compilation succeeded)
```

```
$ ./avg 10 20 30
Average = 20
```

Observation: (10 + 20 + 30) / 3 = 20.

Activity 3 – Maximum and minimum of an array

maxmin.cpp (as given in the manual)

```
#include<iostream>
using namespace std;

int main ()
{
    int arr[10], n, i, max, min;

    cout << "Enter the size of the array : ";
    cin >> n;

    cout << "Enter the elements of the array : ";
    for (i = 0; i < n; i++)
        cin >> arr[i];

    max = arr[0];

    for (i = 0; i < n; i++)
    {
        if (max < arr[i])
            max = arr[i];
    }

    min = arr[0];

    for (i = 0; i < n; i++)
    {
        if (min > arr[i])
            min = arr[i];
    }

    cout << "Largest element : " << max;
    cout << "Smallest element : " << min;

    return 0;
}
```

```
$ g++ maxmin.cpp -o maxmin
(no output means the compilation succeeded)
```

The program asks for the size and then the elements. The input typed was: 5 and then 10 4 25 7 2.

```
$ ./maxmin   (typed: 5, then 10 4 25 7 2)
Enter the size of the array : Enter the elements of the array : Largest element :
25Smallest element : 2
```

Observation: Largest = 25 and smallest = 2, as expected. The two results appear on the same line because the code has no endl or newline after the first cout. Add << endl after << max to print them on separate lines.

Graded Task 1 – Array input from the command line

maxmin_cmd.cpp

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        cout << "Usage: ./maxmin_cmd value1 value2 ..." << endl;
        return 1;
    }

    int max = atoi(argv[1]);
    int min = atoi(argv[1]);

    for (int i = 2; i < argc; i++)
    {
        int v = atoi(argv[i]);
        if (v > max) max = v;
        if (v < min) min = v;
    }

    cout << "Largest element : " << max << endl;
    cout << "Smallest element : " << min << endl;
    return 0;
}
```

```
$ g++ maxmin_cmd.cpp -o maxmin_cmd
(no output means the compilation succeeded)
```

```
$ ./maxmin_cmd 10 4 25 7 2
Largest element : 25
Smallest element : 2
```

Observation: argv[1] to argv[5] hold the five values. The program does not need any cin input.

Graded Task 2 – Palindrome number

palindrome.cpp

```
#include <iostream>
using namespace std;

int main()
{
    int n, original, reverse = 0, digit;
    cout << "Enter number: ";
    cin >> n;

    original = n;

    while (n > 0)
    {
        digit = n % 10;
        reverse = reverse * 10 + digit;
        n = n / 10;
    }

    if (original == reverse)
        cout << "Palindrome";
    else
        cout << "Not Palindrome";

    return 0;
}
```

```
$ g++ palindrome.cpp -o palindrome
(no output means the compilation succeeded)
```

```
$ ./palindrome   (typed: 121)
Enter number: Palindrome
```

Observation: 121 reversed is 121, so it is a palindrome.

```
$ ./palindrome   (typed: 123)
Enter number: Not Palindrome
```

Observation: 123 reversed is 321, so it is not a palindrome.

Graded Task 3 – 1000 random numbers and the prime count

prime.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

bool isPrime(int n)
{
    if (n < 2) return false;

    for (int i = 2; i * i <= n; i++)
```

```
        if (n % i == 0)
            return false;

    return true;
}

int main()
{
    int arr[1000];
    int count = 0;

    srand(time(0));

    for (int i = 0; i < 1000; i++)
        arr[i] = 2 + rand() % 99;

    for (int i = 0; i < 1000; i++)
        if (isPrime(arr[i]))
            count++;

    cout << "Number of prime values = " << count << endl;
    return 0;
}
```

```
$ g++ prime.cpp -o prime
(no output means the compilation succeeded)
```

```
$ ./prime
Number of prime values = 254
```

```
$ ./prime
Number of prime values = 247
```

Observation: The two runs gave different counts (254 and 247) because the numbers are random and srand(time(0)) uses a different seed each second. Your count will also differ.

Quick Summary of Results

Command / program	Result
ps / ps -A / ps -e	Snapshot of processes; PID, TTY, TIME, CMD columns
ps -p 1 -F	Process 1, parent 0 – the first process of the system
top	Live view; PR 20 and NI 0 for normal processes
sleep 300 & ; jobs	Job [1] Running in the background
Ctrl+Z, then bg	Job goes Stopped, then Running again
nice -n 10 / renice	PRI 90 (NI 10) became 85 (NI 5); PRI 65 needs NI -15
g++ hello.cpp -o hello; ./hello	Hello World!

Command / program	Result
./avg 10 20 30	Average = 20
maxmin with 10 4 25 7 2	Largest 25, Smallest 2
palindrome 121 / 123	Palindrome / Not Palindrome
prime.cpp	Count changes each run (254, 247 in these runs)